

EASY LEVEL

1.

```
let a = 10;
```

```
function test() {  
  console.log(a);  
}
```

```
test();
```

Explain:

- Which lexical environment stores `a`?
 - How does `test()` access `a`?
 - Which scope chain lookup happens?
-

2.

```
function outer() {  
  let b = 20;
```

```
  function inner() {  
    console.log(b);  
  }
```

```
  inner();  
}
```

```
outer();
```

Explain:

- Where is `b` stored?
 - Why can `inner()` access `b`?
 - Which lexical environment reference is used?
-

3.

```
let x = 1;
```

```
function demo() {  
  let x = 2;  
  console.log(x);  
}
```

```
demo();  
console.log(x);
```

Explain:

- Why are there two different `x` variables?
 - Which execution context uses which `x`?
-

4.

```
let a = 5;
```

```
function one() {  
  let a = 10;
```

```
  function two() {  
    console.log(a);  
  }  
}
```

```
  two();  
}
```

```
one();
```

Explain:

- Why global `a` is not used?
 - Which lexical environment is searched first?
-

5.

```
let name = "JS";
```

```
function show() {  
  console.log(name);
```

```
}  
  
function change() {  
  let name = "React";  
  show();  
}  
  
change();
```

Explain:

- Why lexical scope depends on where function is defined, not called?
 - Which parent environment belongs to `show()`?
-

6.

```
function test() {  
  let a = 1;  
  
  if (true) {  
    let b = 2;  
    console.log(a);  
    console.log(b);  
  }  
  
  console.log(b);  
}  
  
test();
```

Explain:

- Block scope vs function scope
 - Why `b` behaves differently from `a`
-

7.

```
var x = 10;  
  
function demo() {  
  console.log(x);  
  
  var x = 20;  
}
```

```
demo();
```

Explain:

- Hoisting inside function scope
 - Variable environment creation phase
 - Why local `x` shadows global `x`
-

8.

```
let x = 10;
```

```
function test() {  
  console.log(x);  
  let x = 20;  
}
```

```
test();
```

Explain:

- TDZ behavior
 - Lexical environment creation for `let`
-

9.

```
const a = 100;
```

```
function one() {  
  console.log(a);  
}
```

```
function two() {  
  const a = 200;  
  one();  
}
```

```
two();
```

Explain:

- Why `one()` ignores `two`'s variable

- Lexical parent relationship
-

10.

```
function outer() {  
  let count = 0;  
  
  return function inner() {  
    count++;  
    console.log(count);  
  };  
}
```

```
const fn = outer();
```

```
fn();  
fn();
```

Explain:

- Closure
 - Preserved lexical environment
 - Why `count` does not reset
-

MEDIUM LEVEL

11.

```
let a = 1;
```

```
function first() {  
  let b = 2;
```

```
  function second() {  
    let c = 3;  
    console.log(a, b, c);  
  }  
}
```

```
  second();  
}
```

```
first();
```

Explain:

- Scope chain traversal
 - Nested lexical environments
-

12.

```
function test() {  
  console.log(a);  
}
```

```
let a = 10;
```

```
test();
```

Explain:

- Global lexical environment
 - Why function can access later initialized variable
-

13.

```
let a = 10;
```

```
function outer() {  
  console.log(a);
```

```
  let a = 20;  
}
```

```
outer();
```

Explain:

- Shadowing + TDZ
 - Why outer variable is not used
-

14.

```
var a = 1;
```

```
function test() {  
  console.log(a);  
  
  if (true) {  
    var a = 2;  
  }  
  
  console.log(a);  
}  
  
test();
```

Explain:

- Function scope of `var`
 - Hoisting behavior
-

15.

```
let a = 10;  
  
{  
  let a = 20;  
  
  {  
    let a = 30;  
    console.log(a);  
  }  
  
  console.log(a);  
}  
  
console.log(a);
```

Explain:

- Block lexical environments
 - Shadowing across nested blocks
-

16.

```
function outer() {  
  let x = 10;
```

```
function inner() {  
  console.log(x);  
}
```

```
x = 20;
```

```
return inner;  
}
```

```
const fn = outer();
```

```
fn();
```

Explain:

- Closures store reference or copy?
 - Mutation of outer lexical environment
-

17.

```
function a() {  
  let x = 1;  
  
  return function b() {  
    return function c() {  
      console.log(x);  
    };  
  };  
}
```

```
a()()();
```

Explain:

- Multiple nested closures
 - Lexical environment chaining
-

18.

```
for (var i = 0; i < 3; i++) {  
  setTimeout(() => {  
    console.log(i);  
  }, 0);  
}
```



```
}
```

Explain:

- Single shared lexical environment
 - `var` function scope
 - Callback closure behavior
-

19.

```
for (let i = 0; i < 3; i++) {  
  setTimeout(() => {  
    console.log(i);  
  }, 0);  
}
```

Explain:

- Separate lexical environment per iteration
 - Why `let` behaves differently
-

20.

```
function test() {  
  let a = 10;  
  
  return function () {  
    console.log(a);  
  };  
}
```

```
let a = 50;
```

```
const fn = test();
```

```
fn();
```

Explain:

- Which lexical environment closure uses
 - Why global `a` is ignored
-

HARD LEVEL

21.

```
let a = 1;

function outer() {
  let a = 2;

  return function inner() {
    console.log(a);

    let a = 3;
  };
}

outer();
```

Explain:

- Shadowing + TDZ inside closure
 - Lexical environment creation order
-

22.

```
var a = 10;

function test() {
  console.log(a);

  if (false) {
    var a = 20;
  }
}

test();
```

Explain:

- Hoisting inside block with **var**
 - Function lexical environment
-

23.

```
function outer() {  
  let x = 10;  
  
  return function inner() {  
    console.log(x);  
  
    x++;  
  
    return function deep() {  
      console.log(x);  
    };  
  };  
}  
  
const fn = outer();  
  
const deepFn = fn();  
  
deepFn();
```

Explain:

- Shared closure state
 - Nested closure mutation
-

24.

```
let x = 1;  
  
function a() {  
  console.log(x);  
}  
  
function b() {  
  let x = 2;  
  a();  
}  
  
function c() {  
  let x = 3;  
  b();  
}  
  
c();
```

Explain:

- Why call stack does not affect lexical scope
 - Definition location vs calling location
-

25.

```
function test() {  
  console.log(a);
```

```
  
  {  
    let a = 10;  
  }  
}
```

```
test();
```

Explain:

- Scope lookup failure
 - Block lexical environment visibility
-

26.

```
let x = 100;
```

```
function outer() {  
  return function inner() {  
    console.log(x);  
  };  
}
```

```
let x = 200;  
}
```

```
const fn = outer();
```

```
fn();
```

Explain:

- Function completion before declaration execution
 - Closure parent environment
-

27.

```
function counter() {  
  let count = 0;  
  
  return {  
    increment() {  
      count++;  
      console.log(count);  
    },  
  
    decrement() {  
      count--;  
      console.log(count);  
    }  
  };  
}
```

```
const c = counter();
```

```
c.increment();  
c.increment();  
c.decrement();
```

Explain:

- Shared lexical environment between methods
 - Closure persistence
-

28.

```
var a = 1;
```

```
function test() {  
  console.log(a);
```

```
  function a() {}
```

```
  var a = 10;
```

```
  console.log(a);  
}
```

```
test();
```

Explain:

- Function declaration hoisting priority
 - Variable hoisting
 - Same identifier conflicts
-

29.

```
let a = 1;

function outer() {
  let a = 2;

  function inner() {
    console.log(a);
  }

  a = 5;

  return inner;
}

const fn = outer();

let a = 100;

fn();
```

Explain:

- Closure reference behavior
 - Why updated value is used
-

30.

```
function test() {
  let x = 10;

  return function () {
    console.log(x);

    let x = 20;
  };
}

test()();
```

